Basics

1. Count Digits

Given an integer n, count the total number of digits present in the number.

Example
Input: n = 12345
Digits are 1 , 2 , 3 , 4 , 5 → Output: 5

Approach

Algorithm

1. If n is 0, the number of digits is 1.
2. Initialize a counter as 0.
3. Repeatedly divide n by 10.
4. Increment the counter for each division.
5. Stop when n becomes 0.
6. Return the counter.

Code

```
#include <bits/stdc++.h>

using namespace std;



int countDigits(int n) {

    if (n == 0) return 1;

    int count = 0;

    while (n != 0) {

        count++;

        n /= 10;

    }
```

```
    return count;
}

int main() {
    int n = 12345;
    cout << countDigits(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(\log_{10} n)$, number of digits
- Space Complexity: O(1)

---

2. Reverse a Number

Given an integer n, reverse its digits.

Example
Input: `n = 1234`
Output: `4321`

Approach

Algorithm

1. Initialize `rev = 0`.
2. While n is not `0`:
    - Extract last digit using `n % 10`.
    - Update `rev = rev * 10 + digit`.

- Remove last digit from n using `n /= 10`.

3. Return `rev`.

Code

```
#include <bits/stdc++.h>

using namespace std;



int reverseNumber(int n) {

    int rev = 0;

    while (n != 0) {

        rev = rev * 10 + (n % 10);

        n /= 10;

    }

    return rev;

}



int main() {

    int n = 1234;

    cout << reverseNumber(n);

    return 0;

}
```

Complexity Analysis

- Time Complexity: $O(\log_{10} n)$
- Space Complexity: O(1)

---

3. Check Palindrome

Given an integer n, check whether it is a palindrome.

Example
Input: `n = 121`
Output: `true`

Approach

Algorithm

1. Store the original number.
2. Reverse the number.
3. Compare reversed number with original.
4. If equal, it is a palindrome.

Code

```
#include <bits/stdc++.h>

using namespace std;



bool isPalindrome(int n) {

    int original = n, rev = 0;

    while (n != 0) {

        rev = rev * 10 + (n % 10);

        n /= 10;
```

```
    }

    return rev == original;

}



int main() {

    int n = 121;

    cout << isPalindrome(n);

    return 0;

}
```

Complexity Analysis

- Time Complexity: $O(\log_{10} n)$
- Space Complexity: O(1)

---

4. GCD or HCF

Given two integers a and b, find their GCD (Greatest Common Divisor).

Example
Input: `a = 12, b = 18`
Output: 6

Approach

(Euclidean Algorithm)

Algorithm

1. While b is not 0:

        - a = a % b
        - Swap a and b
    2. When b becomes 0, a is the GCD.

Code

```
#include <bits/stdc++.h>

using namespace std;



int gcd(int a, int b) {

    while (b != 0) {

        int temp = b;

        b = a % b;

        a = temp;

    }

    return a;

}



int main() {

    int a = 12, b = 18;

    cout << gcd(a, b);

    return 0;

}
```

Complexity Analysis

- Time Complexity: O(log min(a, b))
- Space Complexity: O(1)

---

5. Armstrong Numbers

A number is an Armstrong number if the sum of its digits raised to the power of number of digits equals the number itself.

Example
Input: `n = 153`
Explanation: `1³ + 5³ + 3³ = 153`
Output: `true`

Approach

Algorithm

1. Count the number of digits.
2. Traverse each digit.
3. Add `digit^count` to sum.
4. Compare sum with original number.

Code

```
#include <bits/stdc++.h>

using namespace std;



bool isArmstrong(int n) {

    int original = n;

    int digits = log10(n) + 1;
```

```
    int sum = 0;



    while (n != 0) {

        int d = n % 10;

        sum += pow(d, digits);

        n /= 10;

    }

    return sum == original;

}



int main() {

    int n = 153;

    cout << isArmstrong(n);

    return 0;

}
```

Complexity Analysis

- Time Complexity: $O(\log_{10} n)$
- Space Complexity: $O(1)$

---

6. Print All Divisors

Given an integer n, print all its divisors.

Example
Input: `n = 12`
Output: `1 2 3 4 6 12`

Approach

Algorithm

1. Loop from 1 to `sqrt(n)`.
2. If `i` divides `n`:
    - Print `i`
    - If `i != n/i`, print `n/i`.

Code

```
#include <bits/stdc++.h>

using namespace std;



void printDivisors(int n) {

    for (int i = 1; i * i <= n; i++) {

        if (n % i == 0) {

            cout << i << " ";

            if (i != n / i)

                cout << n / i << " ";

        }

    }

}
```

```
int main() {

    int n = 12;

    printDivisors(n);

    return 0;

}
```

Complexity Analysis

- Time Complexity: O(√n)
- Space Complexity: O(1)

---

7. Check for Prime

Given an integer n, check if it is a prime number.

Example
Input: `n = 7`
Output: `true`

Approach

Algorithm

1. If `n ≤ 1`, not prime.
2. Loop from 2 to `sqrt(n)`.
3. If any divisor is found, not prime.
4. Otherwise, prime.

Code

```
#include <bits/stdc++.h>

using namespace std;
```

```
bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0)
            return false;
    }
    return true;
}

int main() {
    int n = 7;
    cout << isPrime(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(√n)
- Space Complexity: O(1)

8. Understand Recursion by Print Something N Times

Given an integer `N`, print a message `"Hello"` exactly `N` times using recursion.

Approach

Algorithm

1. If `N` becomes `0`, stop recursion.
2. Print `"Hello"`.
3. Call the function again with `N-1`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void printHello(int n) {
    if (n == 0) return;
    cout << "Hello\n";
    printHello(n - 1);
}

int main() {
    int n = 3;
    printHello(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N), function is called N times.
- Space Complexity: O(N), recursion stack.

---

9. Print Name N Times Using Recursion

Given an integer `N`, print your name `N` times using recursion.

Approach

Algorithm

1. If `N == 0`, stop.
2. Print the name.

3. Call the function with `N-1`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void printName(int n) {
    if (n == 0) return;
    cout << "Rahul\n";
    printName(n - 1);
}

int main() {
    int n = 4;
    printName(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

10. Print 1 to N Using Recursion

Given an integer `N`, print numbers from 1 to `N` using recursion.

Approach

Algorithm

1. If `N == 0`, stop.
2. First call recursion with `N-1`.
3. Print `N`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void print1toN(int n) {
    if (n == 0) return;
```

```
    print1toN(n - 1);
    cout << n << " ";
}

int main() {
    int n = 5;
    print1toN(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

11. Print N to 1 Using Recursion

Given an integer `N`, print numbers from `N` to 1 using recursion.

Approach

Algorithm

1. If `N == 0`, stop.
2. Print `N`.
3. Call recursion with `N-1`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void printNto1(int n) {
    if (n == 0) return;
    cout << n << " ";
    printNto1(n - 1);
}

int main() {
    int n = 5;
    printNto1(n);
    return 0;
```

```
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

12. Sum of First N Numbers

Given an integer `N`, find the sum of first `N` natural numbers using recursion.

Approach

Algorithm

1. If `N == 0`, return `0`.
2. Return `N + sum(N-1)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

int sumN(int n) {
    if (n == 0) return 0;
    return n + sumN(n - 1);
}

int main() {
    int n = 5;
    cout << sumN(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

13. Factorial of N Numbers

Given an integer `N`, calculate `N!` using recursion.

Approach

Algorithm

1. If `N == 0`, return `1`.
2. Return `N * factorial(N-1)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

int main() {
    int n = 5;
    cout << factorial(n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

14. Reverse an Array

Given an array, reverse it using recursion.

Approach

Algorithm

1. Use two pointers: `l` and `r`.

2. If `l >= r`, stop.
3. Swap `arr[l]` and `arr[r]`.
4. Call recursion with `l+1` and `r-1`.

Code

```
#include <bits/stdc++.h>
using namespace std;

void reverseArray(vector<int>& arr, int l, int r) {
    if (l >= r) return;
    swap(arr[l], arr[r]);
    reverseArray(arr, l + 1, r - 1);
}

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    reverseArray(arr, 0, arr.size() - 1);
    for (int x : arr) cout << x << " ";
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

15. Check if a String is Palindrome or Not

Given a string, check whether it is a palindrome using recursion.

Approach

Algorithm

1. Compare first and last characters.
2. If not equal, return false.
3. Call recursion for the inner substring.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
bool isPalindrome(string &s, int l, int r) {
    if (l >= r) return true;
    if (s[l] != s[r]) return false;
    return isPalindrome(s, l + 1, r - 1);
}

int main() {
    string s = "madam";
    cout << isPalindrome(s, 0, s.size() - 1);
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
- Space Complexity: O(N)

---

16. Fibonacci Number

Given an integer `N`, find the Nth Fibonacci number using recursion.

Approach

Algorithm

1. If `N <= 1`, return `N`.
2. Return `fib(N-1) + fib(N-2)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n = 6;
    cout << fibonacci(n);
```

```
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(2^N), due to repeated calls.
- Space Complexity: O(N), recursion stack.

17. Hashing Theory

Hashing is a technique used to **store and retrieve data efficiently**.
It maps data (keys) to a fixed-size array using a **hash function**, which allows fast access.

The main idea is:

- Convert a value into an index using a hash function
- Store the value at that index
- Retrieve it in constant time

Hashing is mainly used for **frequency counting**, **searching**, and **lookup operations**.

Example
Array: `[1, 2, 2, 3, 1]`
We want to know how many times each number appears.

Using hashing:

- `1 → 2 times`
- `2 → 2 times`
- `3 → 1 time`

Approach

Algorithm

1. Create a hash table (array, map, or unordered_map).

2. Traverse the input data.
3. For each element:
    - Use it as a key in the hash table.
    - Increase its count.
4. The hash table now stores frequency or presence information.

Hashing helps avoid nested loops and reduces time complexity.

Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
   int arr[] = {1, 2, 2, 3, 1};
   int n = 5;

   unordered_map<int, int> freq;

   for (int i = 0; i < n; i++) {
      freq[arr[i]]++;
   }

   for (auto it : freq) {
      cout << it.first << " -> " << it.second << endl;
   }
   return 0;
}
```

Complexity Analysis

- Time Complexity: O(N) average case
- Space Complexity: O(N) for hash table

---

18. Counting Frequencies of Array Elements

Given an array of integers, count the **frequency of each element**.

Example
Input: `[10, 5, 10, 15, 10, 5]`
Output:

- `10 → 3`
- `5 → 2`
- `15 → 1`

Approach

Algorithm

1. Create an unordered_map to store element → frequency.
2. Traverse the array.
3. For each element, increment its count in the map.
4. Print the stored frequencies.

Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
   int arr[] = {10, 5, 10, 15, 10, 5};
   int n = 6;

   unordered_map<int, int> freq;

   for (int i = 0; i < n; i++) {
      freq[arr[i]]++;
   }

   for (auto it : freq) {
      cout << it.first << " occurs " << it.second << " times" << endl;
   }
   return 0;
}
```

- Time Complexity: O(N)

- Space Complexity: O(N)

Reason:

- We traverse the array once.
- Hash map stores at most N elements.

---

19. Find the Highest / Lowest Frequency Element

Given an array, find:

- The element with the **highest frequency**
- The element with the **lowest frequency**

If multiple answers exist, any one is acceptable unless specified.

Example
Input: `[1, 1, 2, 2, 2, 3]`
Highest frequency → 2
Lowest frequency → 3

Approach

Algorithm

1. Count frequencies using hashing.
2. Initialize:
    - `maxFreq = -∞`
    - `minFreq = +∞`
3. Traverse the hash map:
    - Update max frequency and corresponding element.
    - Update min frequency and corresponding element.
4. Print both elements.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    int arr[] = {1, 1, 2, 2, 2, 3};
    int n = 6;

    unordered_map<int, int> freq;

    for (int i = 0; i < n; i++) {
        freq[arr[i]]++;
    }

    int maxFreq = INT_MIN, minFreq = INT_MAX;
    int maxElement = -1, minElement = -1;

    for (auto it : freq) {
        if (it.second > maxFreq) {
            maxFreq = it.second;
            maxElement = it.first;
        }
        if (it.second < minFreq) {
            minFreq = it.second;
            minElement = it.first;
        }
    }

    cout << "Highest frequency element: " << maxElement << endl;
    cout << "Lowest frequency element: " << minElement << endl;

    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N)
    - One pass for frequency counting
    - One pass over hash map
- Space Complexity: O(N)
    - Hash map stores frequencies of elements
